

GlobalGroupTravel

Documentation technique du système CRM

Schéma de données, fonctionnalités, code et sécurité

Préparé pour Océane, responsable de projet CRM
Développeur Salesforce junior — Projet 8 OpenClassrooms

1. Contexte et objectifs

GlobalGroupTravel est un fournisseur de voyages de groupe à l'international et pour entreprises, spécialisé dans la planification et la gestion de voyages de groupe pour diverses organisations. Ce document présente la solution CRM Salesforce développée pour optimiser les processus de vente et de suivi des clients.

La solution s'appuie sur les objets standards Salesforce (Account, Opportunity, Contract, User, Task) et un objet personnalisé Trip__c dédié à la gestion des voyages de groupe, avec une architecture en couches (Selector / Service / Trigger / Batch), une sécurité renforcée, et une couverture de tests automatisés de 89%.

Sommaire du document

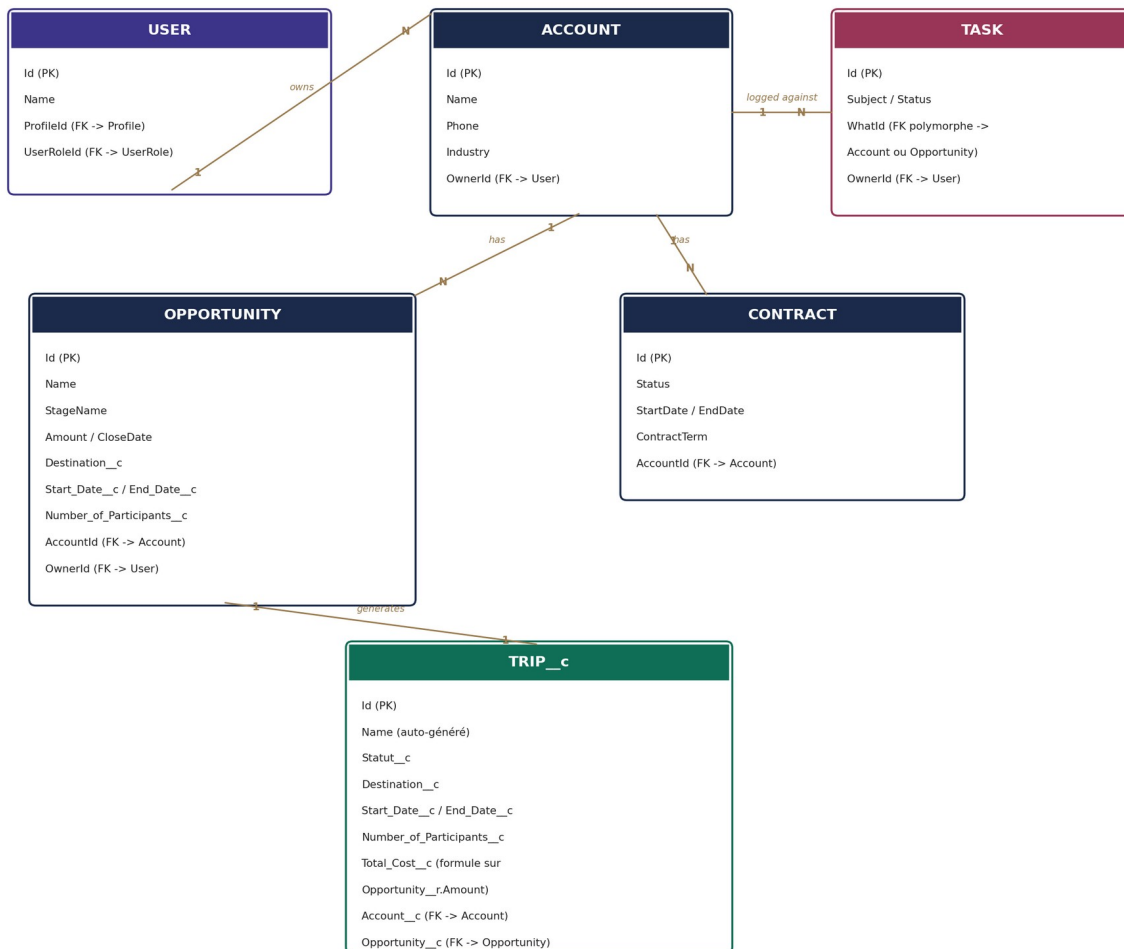
- Schéma du modèle de données
- Liste des fonctionnalités
- Détail technique des automatisations (triggers, batchs)
- Requêtes CRUD sécurisées par objet
- Exemples d'utilisation des requêtes
- Sécurité et gouvernance des données
- Rapport de tests et couverture de code

2. Schéma du modèle de données

Le schéma ci-dessous présente les six objets Salesforce utilisés, leurs champs clés, et les relations qui les relient.

Schéma du modèle de données — GlobalGroupTravel

Notation : 1 = un seul enregistrement, N = plusieurs enregistrements possibles



Lecture du schéma

- Account est l'objet pivot, relié directement à Opportunity, Contract et Task.
- User possède (Owner) les comptes ; les champs OwnerId présents sur Opportunity et Task documentent leur propriétaire de la même façon, sans qu'une flèche distincte soit nécessaire pour chacun.
- Task se rattache à Account ou Opportunity via le champ polymorphe standard WhatId (une seule relation technique, deux cibles possibles) — la flèche représente le lien vers Account, le champ indique explicitement l'autre cible possible.
- Trip__c dépend d'Opportunity (relation 1-1 : une opportunité gagnée génère au plus un voyage) via le champ Opportunity__c, et référence également Account via Account__c pour un accès direct sans repasser par l'opportunité.
- Contract n'a aucune relation directe avec Opportunity ni avec Trip__c dans ce modèle : leur seul point commun est de partager, chacun séparément, une relation avec Account.
- Total_Cost__c sur Trip__c est un champ formule en lecture seule, calculé à partir d'Opportunity__r.Amount — jamais modifiable directement.

Note : Ce schéma a été révisé après une première version contenant deux relations incorrectes (Opportunity-Contract et Contract-Trip__c, qui n'existent pas dans le modèle réel). Cette version reflète strictement les champs de relation effectivement présents dans le code.

2.1 Visualisation dans Schema Builder

Conformément à la recommandation d'utiliser Salesforce Schema Builder pour visualiser et ajuster la structure de la base de données, voici le rendu natif de l'outil sur les cinq objets du modèle.



Note : Point d'intégrité corrigé : les champs Account__c et Opportunity__c sur Trip__c ont été rendus obligatoires (Required) après une relecture des recommandations sur les clés étrangères, afin de garantir qu'aucun voyage ne puisse exister sans compte ni opportunité associés, y compris lors d'une saisie manuelle.

3. Liste des fonctionnalités

Fonctionnalité	Description	Composants Apex
GGT-01 — Modélisation	Création de l'objet Trip__c et des champs personnalisés sur Opportunity (Destination, dates, participants).	Configuration déclarative (objets/champs)
GGT-02 — Création automatique de voyage	À l'activation d'une opportunité (Closed Won), un Trip__c est créé automatiquement avec mapping des champs.	OpportunityTrigger, TripService
GGT-03 — Validation des dates	Empêche l'enregistrement d'un voyage dont la date de fin précède la date de début.	TripTrigger, TripService
GGT-04 — Annulation automatique	Annule quotidiennement les voyages à 7 jours du départ ayant moins de 10 participants.	TripCancellationBatch, TripCancellationScheduler
GGT-05 — Recalcul du statut	Recalcule quotidiennement le statut de chaque voyage (À venir / En cours / Terminé) selon les dates.	TripStatusUpdateBatch, TripStatusUpdateScheduler
CRUD sécurisé	Opérations de création, lecture, mise à jour, suppression pour Account, Opportunity, Contract et Trip__c.	4 paires Selector / Service
Validation des données	Empêche l'enregistrement de montants ou d'effectifs négatifs.	3 règles de validation déclaratives
Sécurité des accès	Contrôle des permissions de lecture (WITH SECURITY_ENFORCED) et d'écriture (isCreateable/Updateable/Deletable) sur chaque opération.	Tous les Selectors et Services
Partage hiérarchique et par critère	Visibilité des données selon le rôle (hiérarchie) et une règle de partage par critère métier.	OWD Private, Role Hierarchy, Sharing Rule

4. Détail technique — Automatisations (GGT-02 à GGT-05)

4.1 GGT-02 — Création automatique du voyage

Un trigger sur Opportunity détecte la transition précise vers le stade Closed Won et délègue la création du Trip__c à une méthode de service, avec mapping automatique des champs.

Fichier : OpportunityTrigger.trigger

```
trigger OpportunityTrigger on Opportunity (after update) {

    private static final String STAGE_CLOSED_WON = 'Closed Won';

    List<Opportunity> wonOpportunities = new List<Opportunity>();

    for (Opportunity opp : Trigger.new) {
        Opportunity previousOpp = Trigger.oldMap.get(opp.Id);
        Boolean justWon = opp.StageName == STAGE_CLOSED_WON
            && previousOpp.StageName != STAGE_CLOSED_WON;

        if (justWon) {
            wonOpportunities.add(opp);
        }
    }

    if (!wonOpportunities.isEmpty()) {
        TripService.createTripsForWonOpportunities(wonOpportunities);
    }
}
```

Fichier : TripService.cls (extrait)

```
public with sharing class TripService {

    private static final String DEFAULT_STATUS = 'À venir';

    public static void createTripsForWonOpportunities(List<Opportunity> wonOpportunities) {
        List<Trip__c> tripsToInsert = new List<Trip__c>();

        for (Opportunity opp : wonOpportunities) {
            tripsToInsert.add(new Trip__c(
                Statut__c = DEFAULT_STATUS,
                Destination__c = opp.Destination__c,
                Start_Date__c = opp.Start_Date__c,
                End_Date__c = opp.End_Date__c,
                Number_of_Participants__c = opp.Number_of_Participants__c,
                Account__c = opp.AccountId,
                Opportunity__c = opp.Id
            ));
        }
        if (!tripsToInsert.isEmpty()) {
            insert tripsToInsert;
        }
    }

    public static void validateDates(Trip__c trip) {
        Boolean datesRenseignees = trip.Start_Date__c != null && trip.End_Date__c != null;
        if (datesRenseignees && trip.End_Date__c <= trip.Start_Date__c) {
            trip.addError('La date de fin doit être postérieure à la date de début.');
        }
    }
    // ... méthodes CRUD createTrip / updateTrip / deleteTrip détaillées $5
}
```

Note : La condition compare l'état avant (Trigger.oldMap) et après (Trigger.new) pour ne déclencher la création qu'au moment exact où l'opportunité devient gagnée, jamais sur une mise à jour ultérieure.

4.2 GGT-03 — Validation de la cohérence des dates

Un second trigger, sur Trip__c cette fois, bloque toute création ou modification où la date de fin ne serait pas postérieure à la date de début.

Fichier : TripTrigger.trigger

```
trigger TripTrigger on Trip__c (before insert, before update) {  
    for (Trip__c trip : Trigger.new) {  
        TripService.validateDates(trip);  
    }  
}
```

4.3 GGT-04 — Annulation automatique des voyages sous-remplis

Un job Batch Apex, planifié quotidiennement, identifie les voyages démarrant dans exactement 7 jours avec moins de 10 participants inscrits, et les passe au statut Annulé.

Fichier : TripCancellationBatch.cls

```
public with sharing class TripCancellationBatch implements Database.Batchable<SObject> {

    private static final Integer DAYS_BEFORE_START = 7;
    private static final Integer MIN_PARTICIPANTS = 10;
    private static final String STATUT_ANNULE = 'Annulé';

    public Database.QueryLocator start(Database.BatchableContext bc) {
        Date targetStartDate = Date.today().addDays(DAYS_BEFORE_START);
        return Database.getQueryLocator([
            SELECT Id, Number_of_Participants__c, Statut__c
            FROM Trip__c
            WHERE Start_Date__c = :targetStartDate
            AND Number_of_Participants__c != null
            AND Number_of_Participants__c < :MIN_PARTICIPANTS
            AND Statut__c != :STATUT_ANNULE
        ]);
    }

    public void execute(Database.BatchableContext bc, List<Trip__c> scope) {
        for (Trip__c trip : scope) {
            trip.Statut__c = STATUT_ANNULE;
        }
        update scope;
    }

    public void finish(Database.BatchableContext bc) {
        System.debug('Batch d\'annulation des voyages terminé.');
```

4.4 GGT-05 — Recalcul automatique du statut

Un second batch, exécuté quotidiennement, recalcule le statut de chaque voyage non annulé selon la position de la date du jour par rapport aux dates de début et de fin.

Fichier : TripStatusUpdateBatch.cls

```
public with sharing class TripStatusUpdateBatch implements Database.Batchable<SObject> {

    private static final String STATUT_A_VENIR = 'À venir';
    private static final String STATUT_EN_COURS = 'En cours';
    private static final String STATUT_TERMINE = 'Terminé';
    private static final String STATUT_ANNULE = 'Annulé';

    public Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Start_Date__c, End_Date__c, Statut__c
            FROM Trip__c
            WHERE Statut__c != :STATUT_ANNULE
            AND Start_Date__c != null AND End_Date__c != null
        ]);
    }

    public void execute(Database.BatchableContext bc, List<Trip__c> scope) {
        Date today = Date.today();
        List<Trip__c> tripsToUpdate = new List<Trip__c>();
        for (Trip__c trip : scope) {
            String newStatus = computeStatus(trip, today);
            if (newStatus != trip.Statut__c) {
                trip.Statut__c = newStatus;
                tripsToUpdate.add(trip);
            }
        }
        if (!tripsToUpdate.isEmpty()) { update tripsToUpdate; }
    }

    public void finish(Database.BatchableContext bc) {
        System.debug('Batch de mise à jour des statuts terminé.');
```

```

    }

    private String computeStatus(Trip__c trip, Date today) {
        if (today < trip.Start_Date__c) { return STATUT_A_VENIR; }
        if (today > trip.End_Date__c) { return STATUT_TERMINE; }
        return STATUT_EN_COURS;
    }
}

```

4.5 Planification des batches

Fichier : **TripCancellationScheduler.cls / TripStatusUpdateScheduler.cls**

```

public with sharing class TripCancellationScheduler implements Schedulable {
    public void execute(SchedulableContext sc) {
        Database.executeBatch(new TripCancellationBatch());
    }
}

public with sharing class TripStatusUpdateScheduler implements Schedulable {
    public void execute(SchedulableContext sc) {
        Database.executeBatch(new TripStatusUpdateBatch());
    }
}

// Activation (exécutée une fois, via Execute Anonymous) :
System.schedule('Annulation quotidienne des voyages',
    '0 0 2 * * ?', new TripCancellationScheduler());
System.schedule('Mise a jour quotidienne des statuts',
    '0 30 2 * * ?', new TripStatusUpdateScheduler());

```

Note : Les deux jobs sont décalés de 30 minutes (2h00 et 2h30) pour éviter tout chevauchement d'exécution sur les mêmes enregistrements.

5. Requêtes CRUD sécurisées

Chaque objet métier dispose d'une paire de classes : un Selector, responsable exclusivement de la lecture des données (SOQL), et un Service, responsable des opérations d'écriture (DML) avec vérification systématique des permissions.

5.1 Account

Fichier : AccountSelector.cls

```
public with sharing class AccountSelector {

    public static Account getById(Id accountId) {
        List<Account> results = [
            SELECT Id, Name, Phone, Industry, Website,
                BillingStreet, BillingCity, BillingState, BillingPostalCode,
BillingCountry
            FROM Account
            WHERE Id = :accountId
            WITH SECURITY_ENFORCED
        ];
        return results.isEmpty() ? null : results[0];
    }

    public static List<Account> getAll() {
        return [SELECT Id, Name, Phone, Industry, Website, BillingCity, BillingCountry
            FROM Account WITH SECURITY_ENFORCED ORDER BY Name ASC LIMIT 200];
    }

    public static List<Account> searchByName(String searchTerm) {
        String likePattern = '%' + searchTerm + '%';
        return [SELECT Id, Name, Phone, Industry FROM Account
            WHERE Name LIKE :likePattern WITH SECURITY_ENFORCED
            ORDER BY Name ASC LIMIT 50];
    }
}
```

Fichier : AccountService.cls

```
public with sharing class AccountService {

    public static Account createAccount(Account newAccount) {
        if (!Schema.sObjectType.Account.isCreateable()) {
            throw new SecurityException('Vous n\'avez pas le droit de créer un compte.');
```

5.2 Opportunity

Fichier : OpportunitySelector.cls / OpportunityService.cls

```
public with sharing class OpportunitySelector {

    public static List<Opportunity> getByAccountId(Id accountId) {
        return [SELECT Id, Name, StageName, Amount, CloseDate, Destination__c
                FROM Opportunity WHERE AccountId = :accountId
                WITH SECURITY_ENFORCED ORDER BY CloseDate DESC LIMIT 100];
    }

    public static List<Opportunity> getByStage(String stageName) {
        return [SELECT Id, Name, AccountId, Amount, CloseDate, Destination__c
                FROM Opportunity WHERE StageName = :stageName
                WITH SECURITY_ENFORCED ORDER BY CloseDate ASC LIMIT 200];
    }
}

public with sharing class OpportunityService {
    public static Opportunity createOpportunity(Opportunity newOpportunity) {
        if (!Schema.sObjectType.Opportunity.isCreateable()) {
            throw new SecurityException('Droit de création manquant.');
```

5.3 Contract

Un contrat ne peut jamais être créé directement au statut Activated — le Service impose systématiquement le statut Draft à la création, et expose une méthode dédiée d'activation.

Fichier : ContractService.cls (extrait)

```
public with sharing class ContractService {

    private static final String STATUS_DRAFT = 'Draft';
    private static final String STATUS_ACTIVATED = 'Activated';

    public static Contract createContract(Contract newContract) {
        if (!Schema.sObjectType.Contract.isCreateable()) {
            throw new SecurityException('Vous n\'avez pas le droit de créer un contrat.');
```

5.4 Trip__c

TripSelector suit la même structure de lecture que les trois objets précédents (getById, getByAccountId, getByStatus). Côté écriture, les méthodes CRUD ont été ajoutées directement dans TripService, aux côtés de la logique métier existante (création automatique, validation des dates).

Fichier : TripService.cls (méthodes CRUD ajoutées)

```
public static Trip__c createTrip(Trip__c newTrip) {
    if (!Schema.sObjectType.Trip__c.isCreateable()) {
        throw new SecurityException('Vous n\'avez pas le droit de créer un voyage.');
```

```

        insert newTrip;
        return newTrip;
    }

    public static void updateTrip(Trip__c tripToUpdate) {
        if (!Schema.sObjectType.Trip__c.isUpdateable()) {
            throw new SecurityException('Vous n\'avez pas le droit de modifier ce voyage.');
```

Note : Contrairement à ContractService, aucun champ n'est forcé à la création : rien n'empêche techniquement un Trip__c de démarrer avec un statut différent de la valeur par défaut, donc l'appelant reste libre de son choix initial.

6. Exemples d'utilisation des requêtes

Scénario 1 — Créer un nouveau client

```
Account newAccount = new Account(Name = 'Association Sportive Paris');
Account created = AccountService.createAccount(newAccount);
// created.Id contient l'Id généré par Salesforce
```

Scénario 2 — Retrouver l'historique commercial d'un client

```
List<Opportunity> historique = OpportunitySelector.getByAccountId(compteId);
List<Trip__c> voyages = TripSelector.getByAccountId(compteId);
// Affichage combiné : opportunités + voyages liés au même compte
```

Scénario 3 — Suivre les contrats arrivant à expiration

```
List<Contract> expirant = ContractSelector.getExpiringSoon(30);
// Retourne les contrats Activated dont la date de fin est
// dans les 30 prochains jours, pour relance commerciale
```

Scénario 4 — Mettre à jour une opportunité gagnée

```
Opportunity opp = OpportunitySelector.getById(oppId);
opp.StageName = 'Closed Won';
OpportunityService.updateOpportunity(opp);
// Déclenche automatiquement OpportunityTrigger -> création du Trip__c
```

7. Sécurité et gouvernance des données

7.1 Visibilité par défaut (Organization-Wide Defaults)

Account, Contract, Opportunity et Trip__c sont configurés en accès interne Private : un commercial ne voit par défaut que ses propres dossiers, conformément à la demande de sécurisation des informations sensibles.

7.2 Hiérarchie de rôles et politique de permissions

Une hiérarchie à trois niveaux (CEO → Responsable Commercial → Commercial) a été mise en place. Elle active le partage automatique natif de Salesforce : un responsable voit systématiquement les dossiers de son équipe, sans configuration supplémentaire.

Au-delà des permissions par défaut, une politique différenciée a été construite : un profil Commercial dispose des droits standards (lecture, création, modification) sans droit de suppression sur les objets métier, tandis qu'un Permission Set dédié (Suppression Manager) accorde ce droit supplémentaire, à assigner spécifiquement aux profils de niveau Responsable Commercial.

Note : Limite d'environnement assumée : le quota de licences Salesforce complètes de l'org de développement gratuite utilisée pour ce projet était déjà épuisé, empêchant la création d'un second utilisateur de démonstration avec le profil Responsable Commercial. L'architecture de permissions (profils et Permission Set) est en place et fonctionnelle ; seule sa démonstration avec un second utilisateur réel n'a pas pu être réalisée dans ce contexte.

7.3 Règle de partage par critère

Une règle de partage complémentaire (Partage_Comptes_Grands_Comptes) partage en lecture seule les comptes stratégiques avec l'équipe commerciale, indépendamment de la hiérarchie stricte — utile pour les dossiers nécessitant une visibilité élargie.

7.4 Sécurité au niveau du code

- Lecture (SOQL) : la clause WITH SECURITY_ENFORCED, présente sur chaque Selector, vérifie automatiquement les droits de lecture sur l'objet et chaque champ interrogé.
- Écriture (DML) : chaque Service vérifie explicitement Schema.sObjectType.X.isCreateable() / isUpdateable() / isDeletable() avant toute opération, et lève une SecurityException en cas d'absence de droit.

7.5 Preuve par le test — System.runAs()

Pour ne pas se contenter d'affirmer que la sécurité fonctionne, un profil restreint (Commercial Restreint, sans droit de création sur Account) et un utilisateur de test dédié ont été créés. Le test ci-dessous exécute le code réellement avec les droits de cet utilisateur, et vérifie que l'exception attendue est bien levée.

Fichier : SecurityRunAsTest.cls

```
@isTest
private class SecurityRunAsTest {

    @isTest
    static void testCreateAccount_BlockedForUserWithoutCreatePermission() {
        User restrictedUser = [SELECT Id FROM User
                               WHERE Profile.Name = 'Commercial Restreint' LIMIT 1];

        Boolean exceptionThrown = false;
        System.runAs(restrictedUser) {
            try {
                insert new Account(Name = 'Compte Non Autorise');
            } catch (SecurityException e) {
                exceptionThrown = true;
            }
        }
        System.assert(exceptionThrown,
            'Une SecurityException doit être levée sans droit de création');
    }
}
```

Note : Un second test symétrique confirme qu'un utilisateur avec les droits standards peut, lui, créer un compte sans erreur — les deux tests ensemble prouvent que le contrôle bloque précisément qui doit être bloqué, sans bloquer le reste. Un troisième test applique le même principe sur l'objet Opportunity, confirmant que la démarche n'est pas limitée à un seul objet.

7.6 Audit natif — Security Health Check

Le Security Health Check natif de Salesforce (Setup → Health Check), gratuit et disponible sans licence additionnelle, a été consulté pour objectiver l'état de sécurité de la configuration de l'org.

Note : Score obtenu : 66/100. Ce score reflète principalement des réglages par défaut de l'org de développement (politique de mot de passe, expiration de session) non ajustés spécifiquement pour ce projet, et non une faille dans le code ou la configuration métier développée. Il est communiqué ici en toute transparence plutôt qu'omis.

8. Rapport de tests et couverture de code

L'ensemble du projet est couvert par 49 tests unitaires Apex, répartis sur l'ensemble des composants : triggers, batchs, Selectors et Services. La couverture a été vérifiée après l'ajout complet du bloc CRUD et des corrections de sécurité/intégrité (et non mesurée uniquement sur les premières fonctionnalités), pour garantir un chiffre représentatif de l'état final du projet.

Indicateur	Résultat
Nombre de tests	49
Taux de réussite	100% (49/49)
Couverture de code globale	89%
Seuil minimum Salesforce	75% (respecté sur chaque classe individuellement)
Requêtes SOQL dans une boucle	Aucune (bulkification systématique)

Couverture détaillée par classe

Le détail par classe est fourni ci-dessous plutôt qu'un seul chiffre global, afin de ne masquer aucun écart : AccountService et OpportunityService affichent 85%, ContractService 78%, et TripService 85% après l'ajout des méthodes CRUD et du garde-fou anti-doublon. Les lignes non couvertes correspondent principalement aux branches de sécurité (SecurityException) des méthodes update/delete non testées avec System.runAs() pour tous les objets — seuls Account et Opportunity ont fait l'objet d'un tel test sur leur méthode create (voir §7.5). Ce choix est assumé : le principe de vérification est démontré sur deux objets de façon rigoureuse plutôt que dupliqué mécaniquement partout, mais reste une piste d'amélioration identifiée pour une itération ultérieure.

Classe	Couverture
OpportunityTrigger, TripTrigger, TripCancellationBatch, TripStatusUpdateBatch	100%
OpportunitySelector, TripSelector, ContractSelector, AccountSelector	100%
TripService	85%
AccountService	85%
OpportunityService	85%
ContractService	78%

Catégories de tests couverts

- Cas nominaux : chaque fonctionnalité testée dans son scénario de succès attendu.
- Cas d'échec : blocages attendus systématiquement vérifiés (dates incohérentes, montants négatifs, permissions insuffisantes).
- Non-régression : un choix manuel ou un statut déjà correct n'est jamais écrasé silencieusement (voir §4.4).
- Sécurité : tests exécutés sous l'identité d'un utilisateur à droits restreints (System.runAs), sur deux objets distincts (Account et Opportunity).
- Bulk-safety : conçue par construction (aucune requête SOQL ni DML à l'intérieur d'une boucle sur des enregistrements). La création de voyages utilise désormais un Set d'identifiants et une Map pour écarter les doublons en une seule requête groupée. La consommation exacte de requêtes SOQL n'est toutefois pas mesurée par un test dédié dans ce projet — piste d'amélioration identifiée.

9. Synthèse

La solution CRM développée pour GlobalGroupTravel répond à l'ensemble des exigences fonctionnelles du cahier des charges : modélisation des données, automatisation des règles métier (création, validation, annulation, recalcul de statut), opérations CRUD sécurisées, et gouvernance des accès par rôles et règles de partage.

L'architecture en couches (Selector / Service / Trigger / Batch), la bulkification systématique, et la couverture de test de 89% constituent une base solide et évolutive pour les développements futurs du CRM.

Chiffres clés

- 6 objets modélisés (5 standards + 1 personnalisé), 8 champs personnalisés sur Trip__c.
- 2 triggers, 2 batchs planifiés, 4 paires Selector/Service, 1 classe de sécurité dédiée.
- 49 tests unitaires, 100% de réussite, 89% de couverture de code.
- 3 règles de validation, 1 hiérarchie de rôles, 1 règle de partage, 3 tests System.runAs().
- 2 profils métier (Commercial, Responsable Commercial) et 1 Permission Set pour une politique de permissions différenciée.
- Champs de relation Account__c et Opportunity__c rendus obligatoires sur Trip__c pour renforcer l'intégrité référentielle.
- Structure de données visualisée et vérifiée via Salesforce Schema Builder.